SISTEMAS DISTRIBUÍDOS
TALLER 5

PROYECTO FINAL - PARTE II

OBJETIVO: Desplegar la segunda parte del proyecto final de curso, evaluando conocimientos técnicos en manejo de sesiones, HTTP y WS.

En el Taller 1 hiciste que los usuarios pudieran identificarse y entrar a un lobby. Acá agregamos dos cosas grandes:

- **El juego en vivo:** Los jugadores se mueven en un canvas y todos ven a todos en tiempo real. El servidor es la autoridad; el cliente NO calcula su posición.
- **Login con Google:** Además del registro con usuario y contraseña, los usuarios pueden entrar con su cuenta de Google. Ambos métodos producen el mismo JWT y conviven en una sola tabla users.

La idea de este taller es construir el sistema siguiendo los esqueletos que se encontrarán a continuación. Vas a tener que tomar decisiones de diseño, justificarlas en tu README, y demostrar que entiendes por qué cada pieza está donde está.

1. Qué se necesita

Una sola tabla users con un campo provider que indica cómo se autenticó. Esto es importante: si tuvieras dos tablas (users_local, users_google), un mismo username podría tener dos cuentas distintas y todo lo demás del sistema (coordinador, score, replicación futura) tendría que duplicarse.

```sql
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  provider TEXT NOT NULL CHECK(provider IN ('local','google')),
  password_hash TEXT,            -- NULL para usuarios de Google
  google_sub TEXT UNIQUE,        -- NULL para usuarios locales
  email TEXT,                    -- Opcional para local
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

- provider = 'local' → tiene password_hash, no tiene google_sub.
- provider = 'google' → tiene google_sub y email, no tiene password_hash.
- Un mismo username no puede repetirse entre ambos providers.

Luego de definir el esquema de la tabla, tienes libertad para elegir entre dos enfoques para la autenticación (justifica tu elección en el README):

Opción A — Google Identity Services:

1. El cliente carga el botón "Sign in with Google" oficial.
2. El usuario hace clic, Google le devuelve un ID Token (JWT firmado por Google).
3. El cliente hace POST /auth/google con { idToken: "..." }.
4. El servidor verifica el token contra Google (con la librería google-auth-library).
5. Si es la primera vez (google_sub no existe en la base), responde 409 pidiendo un username. El cliente le muestra al usuario un campo "elegí tu username" y reenvía con POST /auth/google con { idToken, username }.
6. Si ya existe, responde 200 con el JWT propio.

Opción B — OAuth 2.0 Authorization Code Flow:

1. El cliente redirige al usuario a Google con client_id, redirect_uri, scope=openid email profile.
2. Google redirige de vuelta al backend con un code.
3. El backend intercambia ese code por un ID token.
4. Mismo flujo de "first-time signup" que en la opción A.

2. Lo qué se va a construir

Paso 1: Auth

Tres endpoints HTTP:

Método	Ruta	Body	Respuesta
POST	/register	{ username, password }	201 { userId, username }
POST	/login	{ username, password }	200 { token, username }
POST	/auth/google	{ idToken, username? }	Ver abajo

Respuestas posibles de /auth/google:

- 200 { token, username } → Ya existía o se creó con éxito.
- 409 { error: "username_required", hint: "..." } → Es la primera vez que ese usuario de Google entra y no mandó username. El cliente debe pedírselo y reintentar.
- 409 { error: "username_taken" } → Mandó un username pero ya existe.
- 401 { error: "invalid_id_token" } → El ID token no es válido.

Validaciones obligatorias:

- El ID token debe verificarse contra Google. No alcanza con decodearlo, hay que validar la firma. Eso es lo que hace google-auth-library con client.verifyIdToken({ idToken, audience: GOOGLE_CLIENT_ID }).
- El audience (campo aud del token) debe coincidir con tu GOOGLE_CLIENT_ID. Si no, alguien podría reusar tokens emitidos para otra app y entrar a la tuya.
- El campo email_verified del payload debe ser true. Si no, no aceptes el login.
- El sub del payload es el identificador estable del usuario en Google. Eso es lo que guardás en google_sub. No uses el email como llave. Los usuarios pueden cambiar el email asociado a su cuenta de Google.

Esqueleto de auth-service/index.js:

```javascript
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const bcrypt = require('bcryptjs');
```

```
const jwt = require('jsonwebtoken');
const { OAuth2Client } = require('google-auth-library');
const { DatabaseSync } = require('node:sqlite');

const PORT = parseInt(process.env.PORT || '4000', 10);
const JWT_SECRET = process.env.JWT_SECRET;
const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN || '1h';
const BCRYPT_ROUNDS = parseInt(process.env.BCRYPT_ROUNDS || '10', 10);
const GOOGLE_CLIENT_ID = process.env.GOOGLE_CLIENT_ID;

if (!JWT_SECRET || JWT_SECRET.length < 32) { /* error y exit */ }
if (!GOOGLE_CLIENT_ID) { /* error y exit */ }

const googleClient = new OAuth2Client(GOOGLE_CLIENT_ID);

const db = new DatabaseSync('users.db');
db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    provider TEXT NOT NULL CHECK(provider IN ('local','google')),
    password_hash TEXT,
    google_sub TEXT UNIQUE,
    email TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
  )
`);

const stmts = {
  insertLocal:    db.prepare("INSERT INTO users (username, provider,
password_hash) VALUES (?, 'local', ?)"),
  insertGoogle:   db.prepare("INSERT INTO users (username, provider,
google_sub, email) VALUES (?, 'google', ?, ?)"),
  findByUsername: db.prepare('SELECT * FROM users WHERE username = ?'),
  findByGoogle:   db.prepare('SELECT * FROM users WHERE google_sub =
?'),
};

const app = express();
app.use(cors());
app.use(express.json({ limit: '4kb' }));
```

```
function emitToken(user) {
  return jwt.sign(
    { userId: user.id, username: user.username, provider: user.provider
},
    JWT_SECRET,
    { expiresIn: JWT_EXPIRES_IN }
  );
}

function validateUsername(u) {
  if (typeof u !== 'string') return 'username debe ser string';
  if (u.length < 3 || u.length > 32) return 'username 3-32 chars';
  if (!/^[a-zA-Z0-9_]+$/.test(u)) return 'username solo letras,
números, _';
  return null;
}

// --- /register (igual que el Taller 1) ---
app.post('/register', async (req, res) => {
  // Validar username y password
  // Hashear con bcrypt
  // stmts.insertLocal.run(username, hash)
  // Manejar SQLITE_CONSTRAINT_UNIQUE -> 409
});

// --- /login (igual que el Taller 1, con un detalle) ---
app.post('/login', async (req, res) => {
  // Buscar usuario, comparar bcrypt
  // IMPORTANTE: si user.provider !== 'local', rechazar.
  //   ese usuario debe entrar con Google, no tiene contraseña.
  // Emitir token con emitToken(user)
});

// --- /auth/google (NUEVO) ---
app.post('/auth/google', async (req, res) => {
  const { idToken, username } = req.body || {};

  if (typeof idToken !== 'string') {
    return res.status(400).json({ error: 'idToken requerido' });
  }

  // 1. Verificar el ID token contra Google
```

```
  let payload;
  try {
    const ticket = await googleClient.verifyIdToken({
      idToken,
      audience: GOOGLE_CLIENT_ID
    });
    payload = ticket.getPayload();
  } catch (err) {
    return res.status(401).json({ error: 'invalid_id_token' });
  }

  if (!payload.email_verified) {
    return res.status(401).json({ error: 'email_not_verified' });
  }

  const googleSub = payload.sub;
  const email = payload.email;

  // 2. ¿Ya existe este usuario de Google?
  const existing = stmts.findByGoogle.get(googleSub);
  if (existing) {
     return res.json({ token: emitToken(existing), username:
existing.username });
  }

  // 3. Es la primera vez. ¿Mandó username?
  if (!username) {
    return res.status(409).json({
      error: 'username_required',
      hint: 'Primer login con Google.'
    });
  }

  const usernameError = validateUsername(username);
  if (usernameError) {
    return res.status(400).json({ error: usernameError });
  }

  // 4. Crear el usuario (puede fallar si el username está tomado)
  try {
    const result = stmts.insertGoogle.run(username, googleSub, email);
```

```
    const newUser = { id: result.lastInsertRowid, username, provider:
'google' };
    return res.json({ token: emitToken(newUser), username });
  } catch (err) {
    if (err.code === 'ERR_SQLITE_ERROR' && err.errcode === 2067) {
      return res.status(409).json({ error: 'username_taken' });
    }
    console.error(err);
    return res.status(500).json({ error: 'internal' });
  }
});

app.listen(PORT, () => console.log(`auth :${PORT}`));
```

Variables de entorno (auth-service/.env):

```
None
PORT=4000
JWT_SECRET=<32+ chars random hex>
JWT_EXPIRES_IN=1h
BCRYPT_ROUNDS=10
GOOGLE_CLIENT_ID=<tu-client-id>.apps.googleusercontent.com
```

Paso 2: Coordinator

El coordinador del Taller 1 más:

- Estado de cada jugador con x, y, intent, extras.
- Game loop a 20 Hz que aplica las intenciones y broadcastea el state.
- Manejo de mensajes intent y extras_update del cliente.

Importante: El servidor es autoritativo. El cliente NO calcula posiciones. Manda intenciones de movimiento ({x:-1|0|1, y:-1|0|1}) y el servidor decide.

El juego debe tener algo conocido como el “Game Loop”, que es básicamente un método que se encarga de verificar constantemente el estado en el que se encuentra el juego.

Esqueleto del game loop:

```javascript
const PLAYER_SPEED = 200; // px/seg
const TICK_RATE = 20;     // Hz
const TICK_MS = 1000 / TICK_RATE;

let lastTick = Date.now();

function tick() {
  const now = Date.now();
  const dt = (now - lastTick) / 1000; // segundos
  lastTick = now;

  for (const p of players.values()) {
    // Normalizar diagonales:
    // Sin esto, los jugadores van ~41% más rápido en diagonal.
    const ix = p.intent.x;
    const iy = p.intent.y;
    const mag = Math.hypot(ix, iy);
    if (mag > 0) {
      p.x += (ix / mag) * PLAYER_SPEED * dt;
      p.y += (iy / mag) * PLAYER_SPEED * dt;
    }

    // Mantener dentro del mundo
    p.x = Math.max(PLAYER_RADIUS, Math.min(WORLD_WIDTH - PLAYER_RADIUS,
p.x));
    p.y = Math.max(PLAYER_RADIUS, Math.min(WORLD_HEIGHT -
PLAYER_RADIUS, p.y));
  }

  broadcast({ type: 'state', t: now, players: snapshot() });
}

setInterval(tick, TICK_MS);
```

Mensajes a manejar en ws.on('message'):

```javascript
ws.on('message', (raw) => {
  let msg;
  try { msg = JSON.parse(raw.toString()); } catch (e) { return; }

  const p = players.get(userId);
  if (!p) return;

  if (msg.type === 'intent') {
    const dir = msg.intent && msg.intent.dir;
    if (!dir || typeof dir.x !== 'number' || typeof dir.y !== 'number')
return;
    // Sanitizar a -1, 0, 1 - NO confiar en lo que mande el cliente
    p.intent = { x: Math.sign(dir.x), y: Math.sign(dir.y) };
    return;
  }

  if (msg.type === 'extras_update') {
    if (!msg.extras || typeof msg.extras !== 'object' ||
Array.isArray(msg.extras)) return;
    if (JSON.stringify(msg.extras).length > 1024) return; // límite
anti-abuso
    p.extras = msg.extras;
    return;
  }
});
```

Mensaje welcome enriquecido:

```javascript
ws.send(JSON.stringify({
  type: 'welcome',
  you: { userId, username },
  world: { width: WORLD_WIDTH, height: WORLD_HEIGHT, playerRadius:
PLAYER_RADIUS, tickRate: TICK_RATE }
}));
```

El cliente necesita el world para inicializar el canvas con las dimensiones correctas.

Inicialización al conectar:

```javascript
players.set(userId, {
  username,
  socket: ws,
  connectedAt: new Date().toISOString(),
  x: PLAYER_RADIUS + Math.random() * (WORLD_WIDTH - 2 * PLAYER_RADIUS),
  y: PLAYER_RADIUS + Math.random() * (WORLD_HEIGHT - 2 * PLAYER_RADIUS),
  intent: { x: 0, y: 0 },
  extras: {}
});
```

Paso 3: Cómo configurar Google Cloud

1. Ve a Google Cloud Console.
2. Crea un proyecto nuevo (o usa uno existente). Le pones un nombre cualquiera, "taller-distribuidos" o lo que sea.
3. En el menú lateral: APIs & Services → OAuth consent screen. Configura:
   - User Type: External.
   - App name, support email: lo que quieras.
   - Scopes: agregá email, profile, openid (con el botón "Add or Remove Scopes").
4. APIs & Services → Credentials → Create Credentials → OAuth client ID:
   - Application type: Web application.
   - Authorized JavaScript origins: agrega http://localhost:3000 (y la URL de ngrok si vas a usarla, en HTTPS).
   - Authorized redirect URIs: si usas opción B, agregá tu callback (ej: http://localhost:4000/auth/google/callback). Si usas la opción A, puedes dejarlo vacío.
5. Te genera un Client ID y un Client Secret. El Client ID es lo que va en GOOGLE_CLIENT_ID. El secret solo lo necesitas si usas la opción B.

Paso 4: Feature Extra

Tienes que implementar al menos un feature replicado adicional usando el mecanismo de extras. La idea: el cliente puede mandar:

```JSON
{ "type": "extras_update", "extras": { "color": "#ff5500" } }
```

El servidor lo guarda en player.extras y lo replica en el próximo tick. Todos los demás clientes lo reciben en el state.

Ejemplos de features:

- **Color:** Color básico del círculo.
- **Estado/animación**: idle, jumping, dancing. El cliente lo renderiza como un anillo, escala, etc.
- **Mood**: Emoji al lado del nombre.
- **Equipo**: Rojos vs Azules.
- **Vida**: Barra debajo del nombre.
- **Provider badge**: Muestra un ícono distinto para usuarios de Google vs locales (ya tienes esa info en el JWT).

Importante: Tienes libertad. Lo único: que sea replicado (lo que un cliente cambia, otros lo ven) y persistente durante la sesión (no es solo un evento puntual).

3. Errores típicos

1. **Olvidarse de normalizar diagonales.** Sin Math.hypot, los jugadores van 41% más rápido en diagonal.

2. **Mover al jugador en ws.on('message').** No. Ahí solo guardás p.intent. Quien mueve es el game loop. Si moviera el handler, la velocidad dependería de cuántos mensajes manda el cliente.

3. **No usar dt en el game loop.** Si simplemente sumas velocidad * 1 cada tick, la velocidad real depende del TICK_RATE. Calcula dt con Date.now().

4. **Hacer broadcast en cada intent.** No. Broadcast lo hace el game loop, una vez por tick.

5. **Confiar en lo que manda el cliente.** Sin validar intent.dir, alguien con DevTools manda {x: 9999} y se teleporta. Sin validar tamaño de extras, alguien manda 10MB y rompe la memoria.

6. **Decodear el ID token de Google sin verificarlo.** Crítico de seguridad. Cualquiera puede generar un JSON con el formato correcto. Hay que verificar la firma con verifyIdToken (que valida contra las claves públicas de Google).

7. **No setear el audience al verificar.** Sin audience, aceptas tokens emitidos para otras apps de Google.

8. **Usar email como llave del usuario de Google.** No. Usa sub. El email puede cambiar.

9. **Tener dos secrets distintas en auth y coordinator.** Los tokens no validan, todo se cae.

10. **Subir el .env al repo.** Tenlo en .gitignore desde el primer commit.

11. **Usar comillas dobles para strings literales en SQL.** SQLite es estricto: WHERE provider = "local" falla con no such column: "local". Usa comillas simples: WHERE provider = 'local'.

4. Entrega y sustentación

Qué se entrega:

1. **Link al repo de GitHub**. Commits incrementales, no uno solo.

2. **README completo**.

3. **Sistema corriendo con túneles en ngrok durante la sustentación.**

Qué se sustenta:

1. Demo del juego: dos pestañas, te movés en una y mostrás que la otra ve el movimiento en vivo. Mostrás tu feature extra activándose en una pestaña y reflejándose en la otra (3 min).

2. **Demo de los dos providers:** Registras un usuario alice con password. Luego, en otra pestaña, te logueas con Google y eliges el username bob_google. Los dos entran al juego y se ven entre sí. Mostrás que alice no puede entrar a /auth/google y que un usuario de Google no puede usar /login con password (3 min).

3. **Demo de autoridad:** Abres DevTools, modificas lastState (la variable global del cliente) para teleportar tu jugador. Mostrás que en la otra pestaña no pasa nada. Tu trampa local no afecta a los demás. Esto demuestra que el servidor es la fuente de verdad (2 min).

4. **Tour del código:** Muestras el game loop, dónde validas los inputs, cómo verificas el ID token de Google, dónde guardas los extras (4 min).

5. Responde preguntas (3 min).